FSBM

Université Hassan II - Casablanca

Faculté des Sciences Ben M'sik

Département mathématique et informatique

Structures de Données

Stuctures de données ?

Enregistrement:

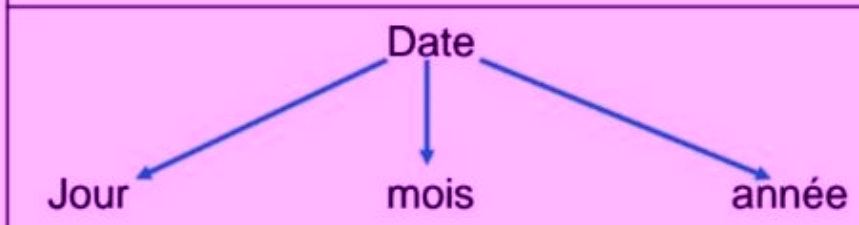
Enregistrement:

Definition:

Les enregistrements sont des structures de données dont les

Enregistrement :

Exemple:

Type structuré (nouveau type)	Variable enregistrement(Variable de ce type)
	D1: Date

Enregistrement: Définition

Jusqu'à présent, nous n'avons utilisé que des types primitifs (caractères, booleen, entiers, réels, chaînes) et des tableaux de types primitifs. Mais il est possible de **créer nos propres types**, puis de déclarer des variables ou des tableaux d'éléments de ce type.

Pour ce faire, il faut déclarer **un nouveau type**, fondé sur d'autres types existants. Après l'avoir défini, on peut alors utiliser **ce type structuré** comme tout autre type normal en déclarant une ou plusieurs variables de ce type. Les variables de type structuré sont appelées **enregistrements**.

Enregistrement: Définition

La déclaration des types structurés se fait, dans les algorithmes, avant la déclaration des variables.

Enregistrement : Déclaration

Une fois qu'on a défini un type structuré, on peut déclarer des variables enregistrements exactement de la même façon que l'on déclare des variables d'un type primitif.

Syntaxe

Nom Var : Nom Enregistrement

Exemple :

S1 : Stagiaire

Enregistrement: Accès aux champs d'un enregistrement

Les champs des enregistrements sont accessibles à travers leur nom, grâce à l'opérateur '.'

Nom Var.NomChamp représente la valeur mémorisée dans le champ de l'enregistrement

Par exemple, pour accéder au groupe de la variable S1, on utilise l'expression : S1.groupe

Remarque :

la lecture d'une telle expression se fait de droit à gauche : le groupe du stagiaire S1.

Enregistrement: Accès aux champs d'un enregistrement

Attention : le nom d'un champ est **TOUJOURS** précédé du nom de la variable déclarée avec le type enregistrement auquel il appartient. On ne peut pas trouver un nom de champ tout seul, sans indication de la variable.

Exercice:

Écrire un algorithme qui permet la saisie des données (nom, prénom et âge) concernant les personnes pers1 et pers2, puis affichage de la différence d'âge entre ces deux personnes

Enregistrement: Accès aux champs d'un enregistrement

Solution: Algorithme DifferenceAge
Début
    // Déclaration de la structure Personne
    Type Personne = Enregistrement
        nom : Chaîne
        prenom : Chaîne
        age : Entier
    Fin Enregistrement

Enregistrement: Accès aux champs d'un enregistrement

Enregistrement: Accès aux champs d'un enregistrement

Solution:
```
// Saisie des informations de la deuxième personne
    Ecrire "Entrez le nom de la deuxième personne : "
    Lire pers2.nom
    Ecrire "Entrez le prénom de la deuxième personne : "
    Lire pers2.prenom
    Ecrire "Entrez l'âge de la deuxième personne : "
    Lire pers2.age
// Calcul de la différence d'âge
    difference ← abs(pers1.age - pers2.age)
// Affichage des informations et de la différence d'âge
    Ecrire "La différence d'âge entre ", pers1.prenom, " et ", pers2.prenom, " est
de ", difference, " ans"

Algorithme
```

Enregistrement: Cas particuliers (Un champ de type structuré)

Un type structuré peut être utilisé comme type pour des champs d'un autre type structuré.

Exemple : Déclarer une structure Personne qui contient les champs : nom , prenom et Date de Naissance (composée du jour, mois, année).

Remarque:

Une date correspond donc à une entité du monde réel qu'on doit représenter par un type enregistrement à 3 champs. Si on déclare le type date au préalable, on peut l'utiliser dans la déclaration du type personne pour le type de la date de

Enregistrement: Cas particuliers (Un champ de type structuré)

Exercice :

Définir la structure **fournisseur** et la structure **Produit** tel que:

- Un fournisseur est caractérisé par son code, sa raison sociale et son numéro de téléphone.
- Un produit est caractérisé par son code, son libellé, et son prix. Aussi, Un produit est livré par un seul fournisseur. Afficher ensuite, le numéro de téléphone du fournisseur d'un produit p.

Enregistrement: Cas particuliers (Un champ de type tableau)

Il est possible aussi qu'un champ de type structuré soit de type **tableau**.

Exercice : Créer une structure ville qui contient les champs suivant (altitude, type de climat, …). Certains de ces critères peuvent être mémorisés pour chaque mois de l'année (par exemple température moyenne, …).

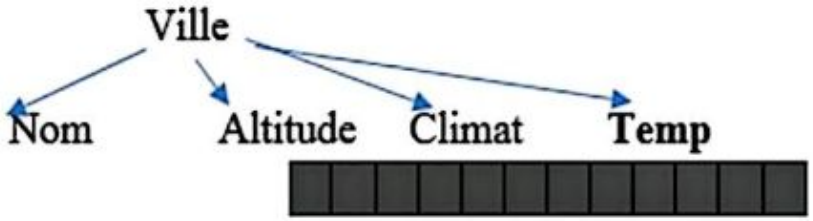
1- Donner la définition de la structure Ville
2- déclarer une ville MaVille et initialiser les infos de la variable MaVille

Enregistrement: Cas particuliers (Un champ de type tableau)

Solution:

```
Ville : enregistrement

Nom : chaîne

Altitude : entier

Climat : Chaîne

Temp : tableau[1..12] de réel
//température moyenne par
mois

Fin enregistrement
```

```
MaVille : Ville
Debut
Ecrire "Donner le nom, le type de climat
et l'altitude de la ville : "
lire MaVille.Nom, MaVille.Climat,
MaVille.Altitude
Ecrire "entrer les températures moyennes
sur les 12 mois "
Pour i allant de 1 à 12 Faire
Lire MaVille.Temp[i]
Fin Pour
Fin
```



Enregistrement: Cas particuliers (Un champ de type tableau)

Enregistrement: Cas particuliers (Tableau de variables structurées)

Il arrive souvent qu'on veut traiter non pas un seul enregistrement mais plusieurs.

Exemple :

On veut traiter un groupe de personne. Donc, On va créer un tableau qui regroupe toutes les personnes du groupe.

Il s'agit alors d'un tableau d'enregistrement.Les colonnes sont appelé es champs et les lignes enregistrements.

Enregistrement: Cas particuliers (Tableau de variables structurées)

Solution:

personne : enregistrement

nom: chaîne

age: entier

Fin enregistrement

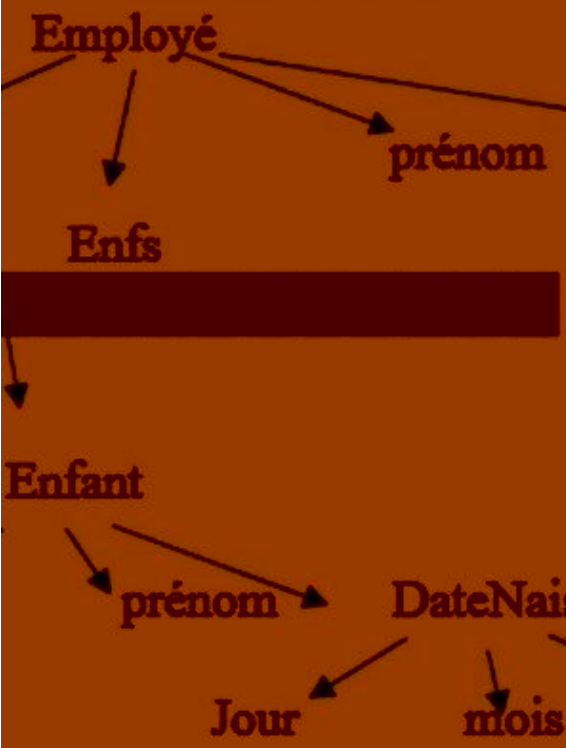
Tableau groupe[20] : personne

Enregistrement: Cas particuliers (Tableau de

Enregistrement: Cas particuliers (Un champ de type tableau d'enregistrements)

: Cas particuliers (Un champ de leau d'enregistrements)

Enregistrement type tab



Enregistrement: Cas particuliers (Un champ de type tableau d'enregistrements)

Enregistrement: Cas particuliers (Un champ de type tableau d'enregistrements)

Structures linéaires. Pile

- **Piles, Files, Listes** font partie des structures dynamiques linéaires.
- Les opérations sur un ensemble dynamique peuvent être regroupées en deux catégories :
  - les requêtes (consultation);
  - les opérations de modification.

  Les opérations classiques : rechercher(S,clé), Insertion(S,elt), Supression(S,elt), Min(S),Max(S), Successeur(S,elt), Prédécesseur(S,elt), etc.

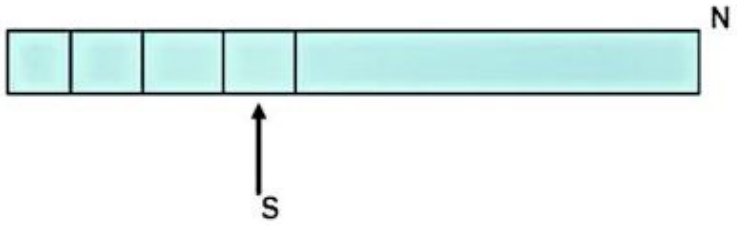
Piles

- Une pile contient des objets insérés et retirés selon le principe du dernier arrivé, premier sorti (last-in-first-out LIFO)
- Les objets peuvent être insérés à tout moment, mais seulement le dernier (le plus récemment inséré) peut être retiré
- Insérer un objet dans la pile correspond à l'empiler (pushing).

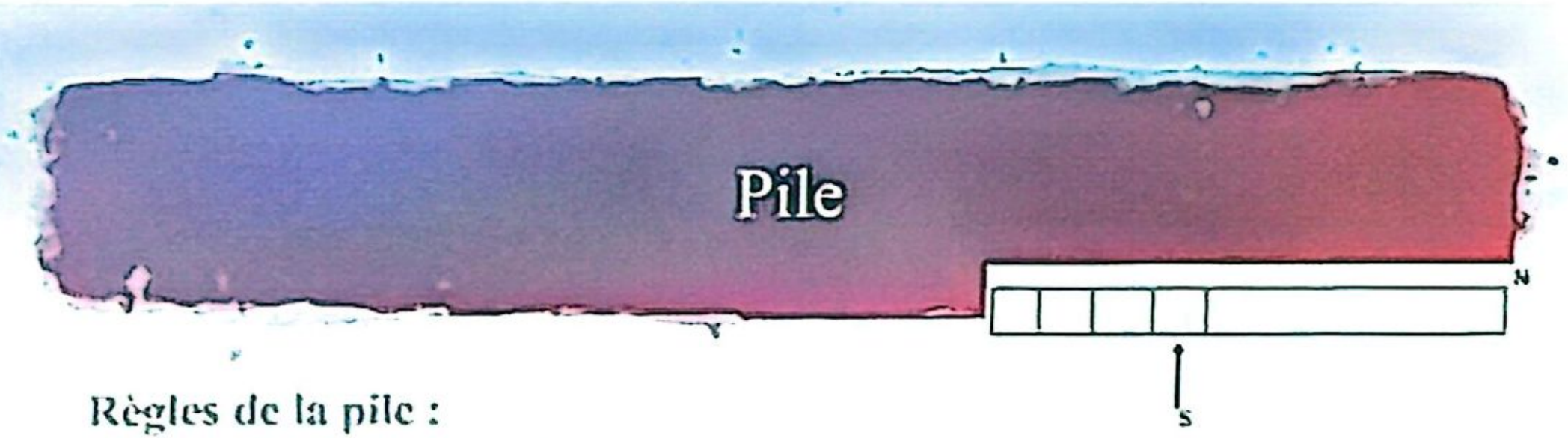
  Dépiler la pile (popping) correspond au retrait d'un objet .
- Analogie: distributeur de bonbons PEZ®

Pile

- Pile est un ensemble dynamique dans lequel l'élément supprimé est celui le plus récemment inséré : dernier entré-premier sorti LIFO
- Opérations :
  - Insérer = Empliler
  - Supprimer = Dépiler
  - Valeur= Recherche



- (toujours l'élément pointé par le sommet)
- Illustration graphique dans le cas d'implémentation par un tableau



Règles de la pile :

- Insertion (push) : On ajoute un élément au sommet de la pile.
- Suppression (pop) : On retire uniquement l'élément en haut de la pile.
- Consultation (top) : On regarde l'élément en haut sans le retirer.

- Re[redacted]

Pile

Insérer un Objet : push() (Empiler)

- L'opération push consiste à ajouter un nouvel élément en haut de la pile.

- Exemple : Ajouter des nombres dans une pile vide
- push(5) → Pile : [5]
- push(10) → Pile : [5, 10]

Pile

Retirer un Objet : pop() (Dépiler)

- L'opération pop consiste à retirer l'élément du sommet de la pile.

Exemple : Retirer des nombres d'une pile

- pop() → Retire 15 → Pile : [5, 10]
- pop() → Retire 10 → Pile : [5]
- pop() → Retire 5 → Pile vide : []

Remarques :

- Si la pile est vide, pop() n'est pas possible (cela provoque une erreur de dépassement appelée underflow).
- On peut toujours consulter l'élément en haut avec top() avant de le dépiler.

Pile

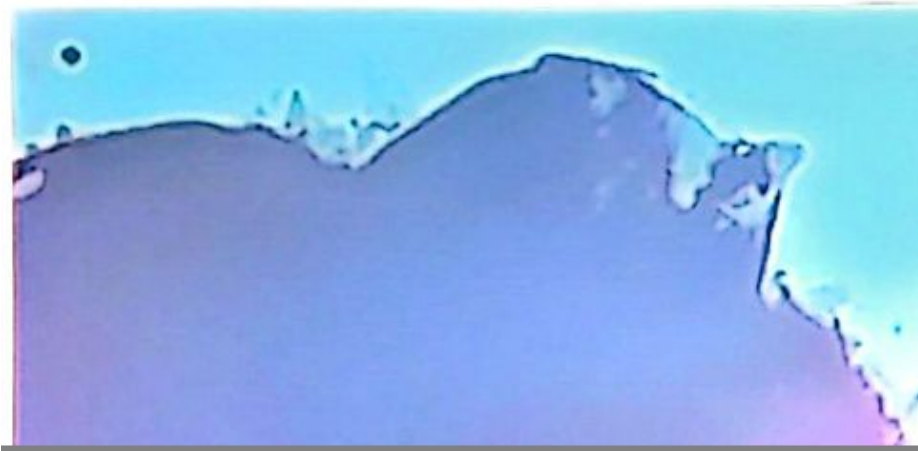
xemples Concrets d'Utilisation des Piles en Informatique

Annuler une action (CTRL + Z)

Quand vous écrivez du texte et que vous annulez une modification (CTRL + Z), le programme revient en

- **Type** Pile
- **utilise** Booléen, Elément
- **Opérations**

Pile-vide	: ->Pile	{création d'une pile vide PV(P)}
Empiler	: Pile x Element ->Pile	{EP(P,a)}
Dépiler	: Pile ->Pile	{supprimer le dernier DP(P)}



Qu'est-ce qu'un TDA (Type de Données Abstrait) ?

Définition : Un Type de Données Abstrait (TDA) est

Exemple de TDA : La Pile

Un TDA Pile définit un ensemble d'opérations possibles sur une pile sans dire

Pile, Implémentation à l'aide d'un tableau(2)

Pile. Implémentation à l'aide d'un tableau(2)

```
#Include <stdio.h>
```

Pile. Implémentation à l'aide d'un tableau(3)

Dépiler un élément de la pile

```
void Depiler(Pile *P) {
 if (EstVide(*P))
```

Pile. Implémentation à l'aide d'un tableau(3)

```
int pile_pleine(Pile P) {
```

Pile. Implémentation à l'aide d'un tableau(3)

Pile Dynamique

namique

Pile Dy

Déclaration d'une pile dynamique

```
Type
    Liste = ^Elem
    Elem = Enregistrement
```

Pile Dynamique

```
**Initialisation de la pile**
void Init_PileD(PileD *P) {
   P->sommet = NULL;}
**Verif
```

Pile Dynamique

```
void DepilerD(PileD *P) {
    if (PileD_Vide(*P)) {
        printf("Erreur : Pile vide !\n");
        return;
    }
    Elem *temp = P->sommet;
    P->sommet = P->sommet->suiv; // Déplacer le sommet
    free(temp); // Libérer la mémoire de l'ancien sommet
}
```

Pile Dynamique

```
// **Retourner la valeur du sommet de la pile**
int ValeurD(PileD P) {
    if (PileD_Vide(P)) {
        printf("Erreur : Pile vide !\n");
```

Les fichiers

➢Motivation

Toutes les structures de données que nous avons vues ont un point commun :

- Résident dans la mémoire principale de l'ordinateur (existence limitée à la durée d'exécution du programme)

⇨Besoin de conserver des données (pour une utilisation future) après l'exécution du programme (exemple : carnet d'adresses)

notion de fichier.

Les fichiers

- Qu'est-ce qu'un fichier?

  Un fichier est une collection de fiches:

Les fichiers

➢ **En informatique :**

Un fichier est une séquence de données du même type enregistrées sur un support informatique (de manière permanente).

- Un fichier est :
  - conservés en *mémoire secondaire*
    (disques et bandes magnétiques, disquettes....)
  - désigné par un nom et possède des attributs tels que date de création, taille,...

Fichiers : organisation et accès

➢ *Définition*

L'organisation d'un fichier est la manière dont sont rangés physiquement les enregistrements du fichier.

- Le but est d'accéder les enregistrements de manière à y accéder le plus

Fichiers : organisation relative (accès direct)

- Chaque enregistrement possède un numéro

Fichiers : organisation indexée

➢**Notion d 'index :**

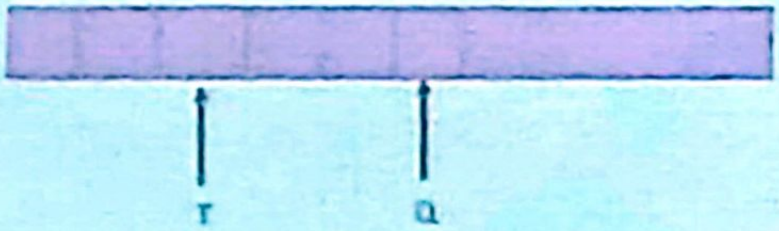
- Soit un fichier F dont les enregistrements E possèdent une clé C (e.g. nom). Un index permet d 'associer à chaque clé C le rang R de l 'enregistrement E de clé C dans F.
- L'index est alors une « table des matières » du fichiers
- On crée des fichiers supplémentaire d'index
- On parcourt un index pour rechercher une clef. On obtient ainsi l'adresse exacte de l'information recherchée.

Les types de fichiers

- On distingue deux catégories:
  - *Les fichiers binaires* contenant du code binaire représentant chaque élément (enregistrement). Ces fichiers ne doivent être manipulés que par des programmes!
  - *Les fichiers textes* (appelés aussi imprimables) contenant des caractères et susceptibles d'être lus, éditées, imprimés...

TDA File(1)

- Une file (queue) est une structure de données linéaire qui suit le principe FIFO (First-In, First-Out), ce qui signifie que le premier élément ajouté est le premier à être retiré.
- Dans le cas d'une file on fait les adjonctions à une extrémité, les accès et les suppressions à l'autre extrémité.
- Les files sont aussi appelées FIFO (first –in –first out) : premier-entré-premier-sorti
- Une file comporte une tête et une queue. Les éléments sont donc retirés et consultés dans la tête et rajoutés dans la queue



Caractéristiques principales d'une file

- **Deux extrémités :**
- Tête (front) : L'endroit où les éléments sont retirés (défilement).
- Queue (rear) : L'endroit où les éléments sont ajoutés (enfilement).
- **Opérations principales :**
- Enfiler (enqueue) : Ajouter un élément à la fin (queue).
- Défiler (dequeue) : Retirer l'élément en tête.
- Front : Accéder à l'élément en tête sans le retirer.
- File vide : Vérifier si la file est vide.
- File pleine (dans le cas d'une file statique).

Types de files

- **File simple (classique)** : Enfilement à la fin, défilement au début.
- **File circulaire** : Gère efficacement l'espace mémoire en réutilisant les cases libérées.
- **File à double extrémité** (Deque - Double Ended Queue) : Enfilement et défilement possibles aux deux extrémités.
- **File de priorité** : Les éléments ont une priorité et sont extraits selon leur importance.

TDA File(2)

- Type File
- utilise Booléen, Elément
- Opérations
  - File-vide : ->File (création d'une file vide FV(F))
  - Enfiler : File x Elément ->File {EF(F,e)}
  - Défiler : File ->File {supprimer le dernier DF(F)}
  - Valeur : File->Elément {renvoie l'élément au sommet sans modifier la file VF(F)}
  - Est-vide : File->Booléen {test si la file est vide EV(F)}

Les opérations Défiler et Valeur ne sont définies que si la file n'est pas vide

Défiler(F) est-défini-ssi est-vide(F)=faux

Valeur(F) est-défini-ssi est-vide(F)=faux

- Axiomes

Est-vide(F)=vrai => Valeur(Enfiler(F,e))=e

Est-vide(F)=faux => Valeur(Enfiler(F,e))=Valeur(F)

Est-vide(F)=vrai => Défiler(Enfiler(F,e))= File-vide

Est-vide(F)=faux => Défiler(Enfiler(F,e))=Enfiler(Défiler(F),e)

Est-vide(File-vide)=vrai

Est-vide(Enfiler(F,e))=faux

File. Implémentation à l'aide d'un tableau(1)

```
#include <stdio.h>
#include <stdlib.h>
#define LMAX 5  // Taille maximale de la file
// Définition de la structure de la file
typedef struct {
    int elements[LMAX];  // Tableau pour stocker les éléments
    int debut;  // Indice du premier élément (tête)
    int queue;  // Indice du dernier élément (queue)
} File;
// **Initialiser la file**
void Init_File(File *F) {
    F->debut = 1;  // La tête commence à l'indice 1
    F->queue = 0;  // La file est vide au départ (queue avant debut)
}
```

File. Implémentation à l'aide d'un tableau(1)

```
// **Vérifier si la file est vide**
int File_Vide(File F) {
    return (F.queue < F.debut);  // Si queue est avant debut, la file est vide
}

// **Vérifier si la file est pleine**
int File_Pleine(File F) {
    return (F.queue == LMAX);  // Si queue atteint LMAX, la file est pleine
}
```

File. Implémentation à l'aide d'un tableau(2)

- **Enfiler :**

```
Procédure Enfiler(réf F: File, val X : Elément)
Début
        Si File_pleine (F) Alors
                Ecrire (''impossible la File est pleine")
        Sinon
                F. Queue ← F.Queue+1;
                F.elts[F. Queue] ← X;
        FinSi
FinEnfiler
```

File. Implémentation à l'aide d'un tableau(2)

```
// **Enfiler un élément (ajouter à la queue)**
void Enfiler(File *F, int val) {
   if (File_Pleine(*F)) {
      printf("Erreur : File pleine !\\n");
      return;
   }
   F->queue++;  // Avancer queue
   F->elements[F->queue - 1] = val;  // Stocker l'élément (indices 0-based en C)
   printf("Enfile : %d\\n", val);
}
```

File. Implémentation à l'aide d'un tableau(2)

- **F->elements[F->queue - 1] = val; fonctionne SI queue est utilisé comme compteur d'éléments.**
- **Une meilleure version est F->elements[F->queue] = val; F->queue++;, qui fonctionne de manière plus intuitive.**
- **Cela évite des erreurs d'indexation et est plus standard en C.**

File. Implémentation à l'aide d'un tableau(3)

- **Défiler:**

```
Procédure Défiler(réf F: File)
Début
        Si File_vide(F) Alors
                Ecrire (''impossible la File est vide")
        Sinon
                F.Début ← F.Début + 1;
                FinSi
FinDéFiler
```

File. Implémentation à l'aide d'un tableau(3)

```
// **Défiler un élément (retirer depuis la tête)**
void Defiler(File *F) {
   if (File_Vide(*F)) {
      printf("Erreur : File vide !\\n");
      return ; }
   printf("Défile : %d\\n", F->elements[F->debut - 1]);
 // Afficher l'élément supprimé
   F->debut++;  // Avancer la tête
   if (F->debut > F->queue) {  // Réinitialisation si la file devient vide
      Init_File(F);}}
```

File. Implémentation à l'aide d'un tableau(3)

```
// **Programme principal**
int main() {
    File maFile;
    Init_File(&maFile);
    // Ajout d'éléments à la file
    Enfiler(&maFile, 10);// queue = 1,   elements[0] = 10
    Enfiler(&maFile, 20); // queue = 2,   elements[1] = 20
    Enfiler(&maFile, 30);// queue = 3,   elements[2] = 30
    Afficher_File(maFile);  // Afficher la file
    // Suppression d'un élément
    Defiler(&maFile);
    Afficher_File(maFile);  // Afficher après suppression
    return 0;}
```

File. Implémentation à l'aide d'un tableau(4)

- **EstVide :**

```
Fonction File_vide (F : File) : Booleen
Debut
        SI F. Début > F. Queue  Alors
                File_vide ← vrais ;
        Sinon
                File_vide ← faux ;
        FinSI ;
Fin
```

- **EstPleine :**

```
Fonction File_pleine (F : File) : Booleen
Debut
        SI (F. Queue = N)  Alors
                File_pleine ← vrais ;
        Sinon
                File_pleine ← faux ;
        FinSI
Fin
```

File Dynamique

Une File dynamique est une liste à laquelle on attache deux (2) pointeurs Debut et Queue. C'est un enregistrement à deux cases : pointeur qui pointe le premier élément de la liste (Debut) et un autre qui pointe la dernière valeur ajoutée dans la liste (Queue).

- Exemples :

1- Une file vide⟶Debut = Queue = Nil

2- Une file a un seul élément⟶Debut = Queue ≠ Nil

3- Une file a plus d'un élément ⟶Debut ≠ Queue

De même que pour les files statiques nous présentons la déclaration et les sous algorithmes de bases détaillés pour le type statique. Et dans un souci d'uniformisation nous utilisons les mêmes noms mais en ajoutant un D (pour rappeler Dynamique) à la fin de chaque nom pour faire la différence

Note : Il n'y a pas de file pleine pour les files dynamique. La seule possibilité de ne pas pouvoir ajouter un élément c'est d'avoir une mémoire pleine, cas que l'on ne prend pas en considération ici.

File Dynamique

les structures

ésente un élément de la file, contenant :

eur stockée.

inteur vers l'élément suivant.

Définition

Elem : repr

info : la va

suiv : un p

File Dynamique

lément de la file

ans l'élément

ers l'élément suivant

ile dynamique

```
// Définition de la structure d'un é
typedef struct Elem {
    int info;        // Valeur stockée d
    struct Elem* suiv; // Pointeur v
} Elem;

// Définition de la structure de la f
typedef struct FileD {
```

File Dynamique

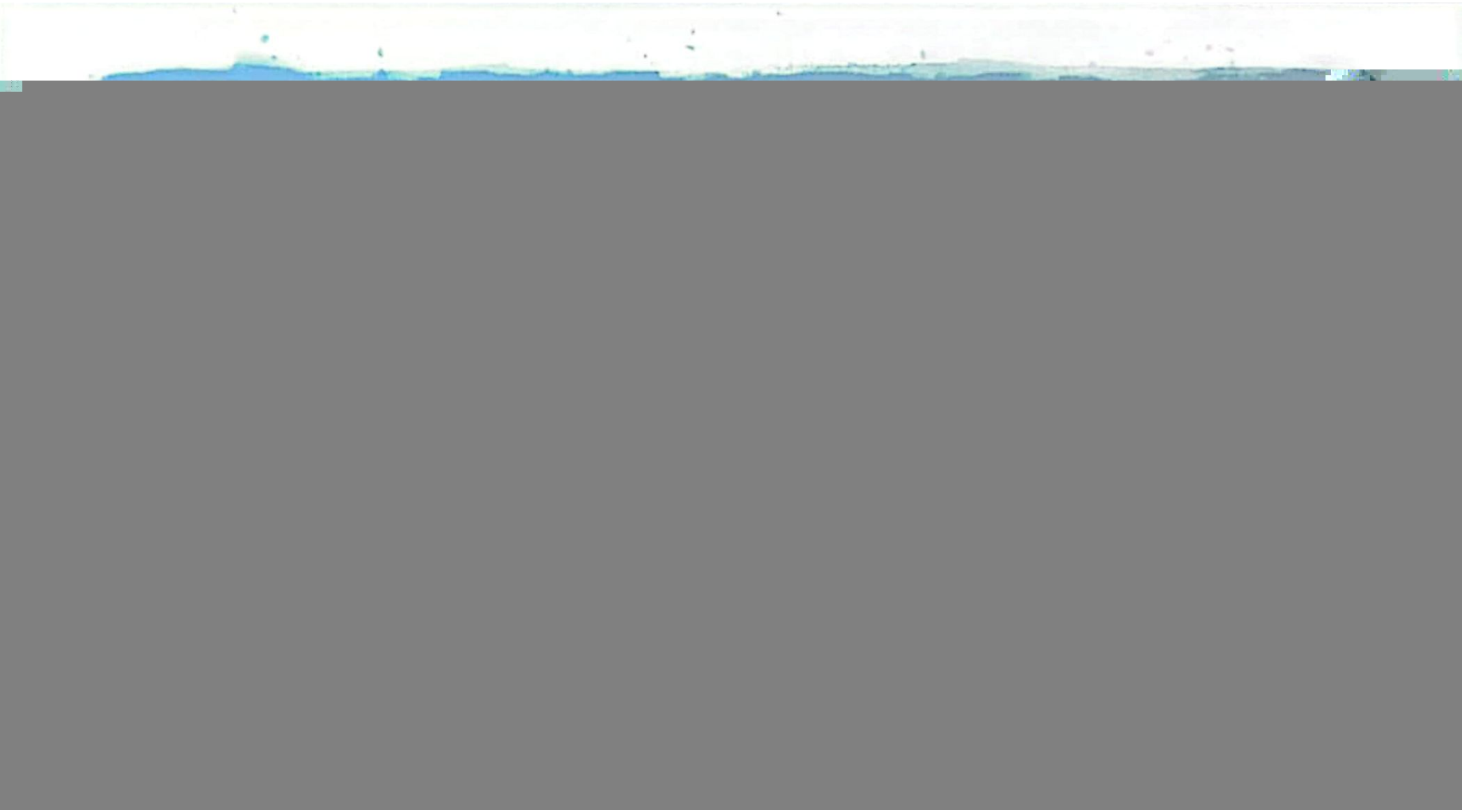
File Dynamique

```
void Init_FileD(FileD *F) {
    F->debut = NULL;
    F->queue = NULL;
}
int FileD_Vide(FileD F) {
    return (F.debut == NULL);
}
```

File Dynamique

```
void DefilerD(FileD *F) {
   if (FileD_Vide(*F)) {
      printf("Erreur : File vide !\\n");
      return; }
   Elem *temp = F->debut;
   printf("Défile : %d\\n", temp->info);
   F->debut = F->debut->suiv;
   if (F->debut == NULL) {
      F->queue = NULL;
   }
   free(temp);
}
```

Listes chaînées(1)



Listes chaînées(1)

Exemple illustratif :

Imaginons une liste contenant les valeurs 5 → 12 → 7 :

Adresse	Valeur	Pointeur vers
@1000	5	@1050
@1050	12	@1080
@1080	7	NULL

La tête pointe vers @1000.

Le premier élément (@1000) contient la valeur 5 et un pointeur vers @1050 (le 2e élément).

Le deuxième pointe vers @1080 (le 3e).

Le dernier (@1080) a un pointeur nul, marquant la fin.

Listes chaînées(1)

Pour accéder au troisième élément :

Suivre la tête : → @1000

Suivre le pointeur : → @1050

Suivre encore le pointeur : → @1080

Listes chaînées(1)

Il existe différents types de listes chainées :

- **Liste chaînée simple** constituée d'éléments reliés entre eux par des pointeurs.
- **Liste chaînée ordonnée** où l'élément suivant est plus grand que le précédent. L'insertion et la suppression d'élément se font de façon à ce que la liste reste triée.
- **Liste doublement chaînée** où chaque élément dispose non plus d'un mais de deux pointeurs pointant respectivement sur l'élément précédent et l'élément suivant. Ceci permet de lire la liste dans les deux sens, du premier vers le dernier élément ou inversement.
- **Liste circulaire** où le dernier élément pointe sur le premier élément de la liste. S'il s'agit d'une liste doublement chaînée alors de premier élément pointe également sur le dernier.

Ces différents types peuvent être mixés selon les besoins. On utilise une liste chaînée plutôt qu'un tableau lorsque l'on

Listes chaînées(2)

Définitions

Un élément d'une liste est l'ensemble (ou structure) formé :

- **d'une donnée ou information,**
- **d'un pointeur nommé Suivant indiquant la position de l'élément le suivant dans la liste. A chaque élément est associée une adresse mémoire.**

Les listes chaînées font appel à la notion de variable dynamique. Une variable dynamique:

- **est déclarée au début de l'exécution d'un programme,**
- **elle y est créée, c'est-à-dire qu'on lui alloue un espace à occuper à une adresse de la mémoire,**
- **elle peut y être détruite, c'est-à-dire que l'espace mémoire qu'elle occupait est libéré,**
- **l'accès à la valeur se fait à l'aide d'un pointeur.**

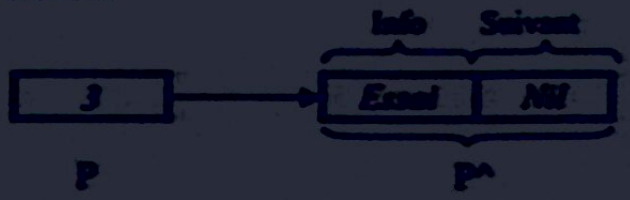
Listes chaînées(2)

Un pointeur est une variable dont la valeur est une adresse mémoire (voir chapitre 9). Un pointeur, noté P, pointe sur une variable dynamique notée P^.

Le type de base est le type de la variable pointée.

Le type du pointeur est l'ensemble des adresses des variables pointées du type de base. Il est représenté par le symbole ^suivi de l'identificateur du type de base.

Exemple:



La variable pointeur P pointe sur l'espace mémoire P^ d'adresse 3. Cette cellule mémoire contient la valeur "Essai" dans le champ Info et la valeur spéciale Nil dans le champ Suivant. Ce champ servira à indiquer quel est l'élément suivant lorsque la cellule fera partie d'une liste. La valeur Nil indique qu'il n'y a pas d'élément suivant. P^ est l'objet dont l'adresse est rangée dans P.

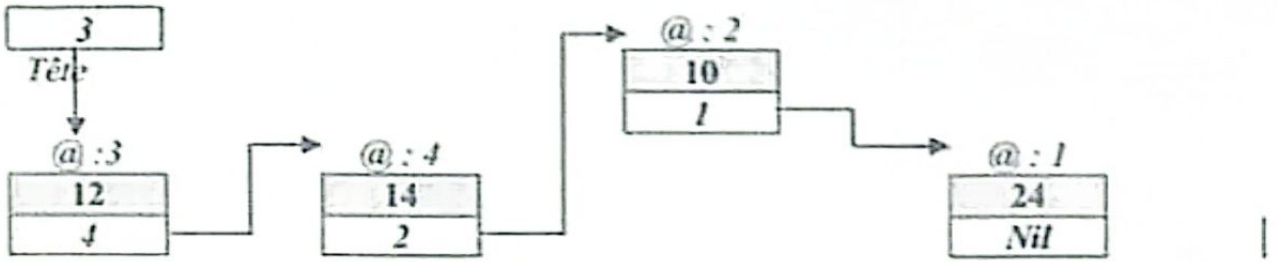
Listes chaînées(2)

Les listes chaînées entraînent l'utilisation de procédures d'allocation et de libération dynamiques de la mémoire. Ces procédures sont les suivantes:

- **Allouer(P):** réserve un espace mémoire P^ et donne pour valeur à P l'adresse de cet espace mémoire. On alloue un espace mémoire pour un élément sur lequel pointe P.
- **Désallouer(P) :** libère l'espace mémoire qui était occupé par l'élément à supprimer P^ sur

Listes chaînées simples

Exemple:



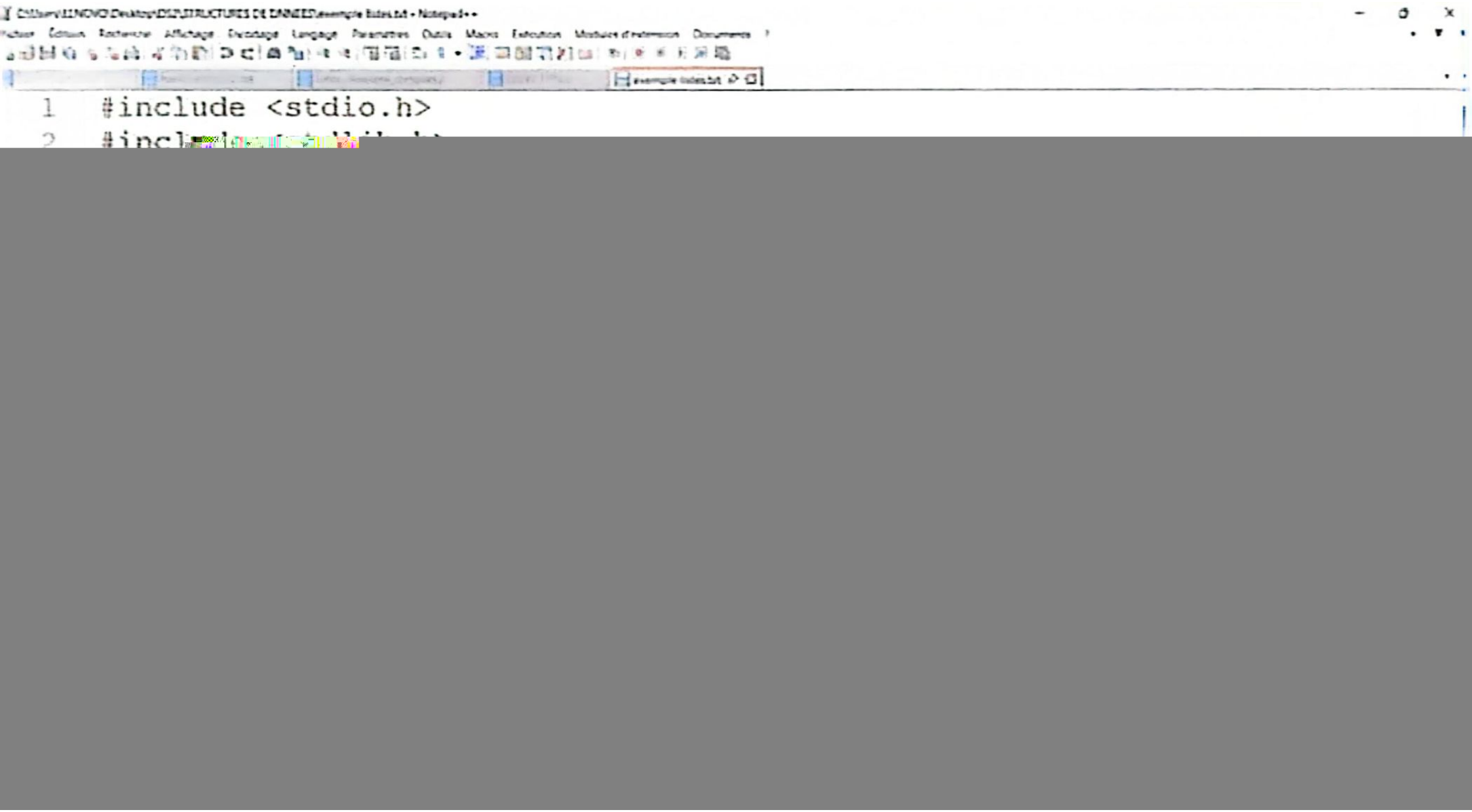
Le 1er élément de la liste vaut 12 à l'adresse 3 (début de la liste chaînée)

Le 2e élément de la liste vaut 14 à l'adresse 4 (car le pointeur de la cellule d'adresse 3 est égal à 4)

Le 3e élément de la liste vaut 10 à l'adresse 2 (car le pointeur de la cellule d'adresse 4 est égal à 2)

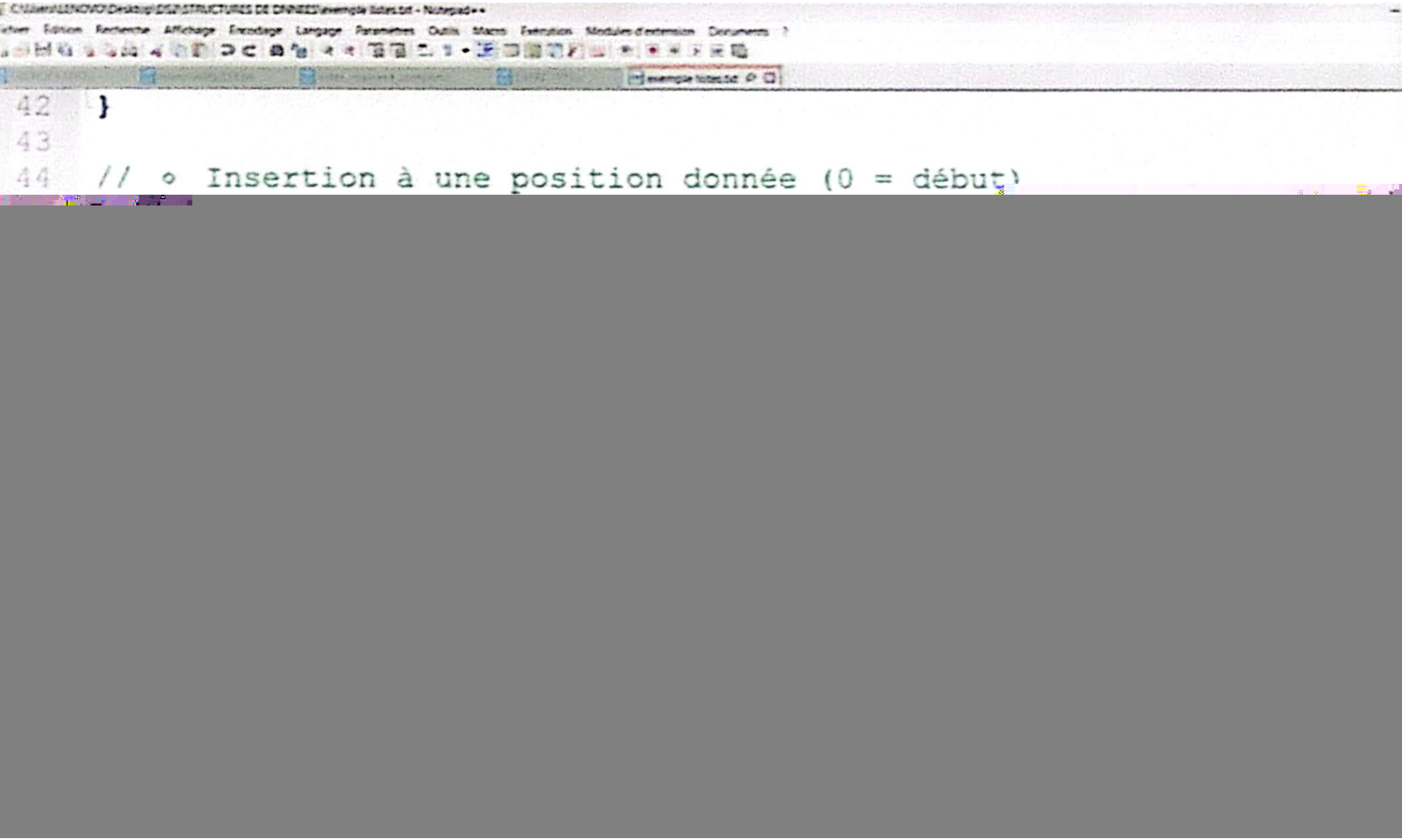
Le 4e élément de la liste vaut 24 à l'adresse 1 (car le pointeur de la cellule d'adresse 2 est égal à 1)

Si P a pour valeur 3 *Si P a pou*


Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents
exemple listes.txt
1 #include <stdio.h>

```
12  // ◦ Fonction pour créer un nouveau nœud
13  Node* createNode(int value) {
14      Node* new_node = (Node*)malloc(sizeof(Node));
15      if (new_node == NULL) {
16          printf("Erreur d'allocation mémoire !\n");
17          exit(1);
18      }
19      new_node->data = value;
20      new_node->next = NULL;
21      return new_node;
22  }
23
24  // ◦ Insertion en début de liste
25  void insertAtBeginning(int value) {
26      Node* new_node = createNode(value);
27      new_node->next = head;
28      head = new_node;
29  }
30
```

```
30
31  // ◦ Insertion à la fin de la liste
32  void insertAtEnd(int value) {
33      Node* new_node = createNode(value);
34      if (head == NULL) {
35          head = new_node;
36          return;
37      }
38      Node* current = head;
39      while (current->next != NULL)
40          current = current->next;
41      current->next = new_node;
42  }
43
44  // ◦ Insertion à une position donnée (0 = début)
45  void insertAtPosition(int value, int position) {
46      if (position == 0) {
            return
```


Fichier Edition Rechercher Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?
exemple listes.txt
42 }
43
44 // ∘ Insertion à une position donnée (0 = début)

```
// ◦ Insertion à une position donnée (0 = début)
void insertAtPosition(int value, int position) {
    if (position == 0) {
        insertAtBeginning(value);
        return;
    }

    Node* new_node = createNode(value);
    Node* current = head;

    for (int i = 0; i < position - 1 && current != NULL; i++)
        current = current->next;

    if (current == NULL) {
        printf("Position invalide !\n");
        free(new_node);
        return;
    }
```

C:\Users\LENOVO\Desktop\DS2\STRUCTURES DE DONNEES\exemple listes.txt - Notepad++